الطابور Queue

في حياتنا اليومية هناك الكثير من الحالات التي نرى فيها الناس يقفون على هيئة طوابير لشراء سلعة او انجاز عمل, هي موجودة غالباً للحصول على عديد الأنواع من الخدمات. تشبه قوائم الانتظار في الحاسبات طوابير الانتظار في الحياة اليومية، ويطبق في كلاهما مبدأ من يأتي أولاً تقدم له الخدمة أولاً، وفي الحاسب يطبق نفس المبدأ .

هيكل الطابور مشابه للمكدس ماعدا امكانية إضافة البيانات تتم من نهاية الطابور والحذف يتم من الامام (بداية الطابور). عند كتابة برنامج الطابور نحتاج إلى عدد إثنين من العدادات التي يتم تمثيله بالأعداد الصحيحة، إحداها يحدد بداية الطابور والثاني يحدد نهايته.

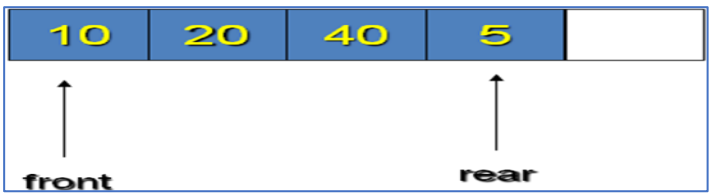
الطابور هو مثال لهياكل البيانات الخطية، هو نوع خاص لتجميع البيانات بحيث أن البيانات في هذا الهيكل يتم ترتيبها ومعالجتها الداخل اولا يخرج أولاً Front In Front Out (FIFO)

بعض تطبيقات الطابور

1. عندما يكون لدينا مصادر مشتركة مثل طابعة تستلم اوامر طباعة من أكثر من حاسوب او تستلم من حاسوب واحد أكثر من أمر طباعة, في هذه الحالة فإن أمر طباعة واحد سيتم تنفيذه بينما الأوامر الاخرى تنتظر دورها للتنفيذ, ويتم تنفيذها حسب اسبقية وصولها أو وفقا لخوارزمية محددة بالتنفيذ.
2. إذا ضغطنا بالماوس وبشكل سريع على أكثر من ملف بفارق وقت قليل جداً، وكان الملف الأول يحتاج بعض الوقت لإنجاز عمله, فان الملف الثاني سيبقى بالانتظار لحين اكمال الملف الاول وعندها يكون بالطابور .
3. داخل الحاسوب فإن الاجهزة المختلفة لها سرعات إنجاز مختلفة، مثلا اجهزة الادخال بطيئة مقارنةً بأجهزة المعالجة لذلك فان عملية ارسال واستلام البيانات بين هذه الأجهزة يؤدي الى توقف بعض الاجهزة السريعة لانتظار البيانات من الاجهزة البطيئة، في مثل هذه الحالة فان هناك عدة طرق لمعالجة الفرق بالسرعة أحدها استخدام الطابور.

الطوابير الخطية

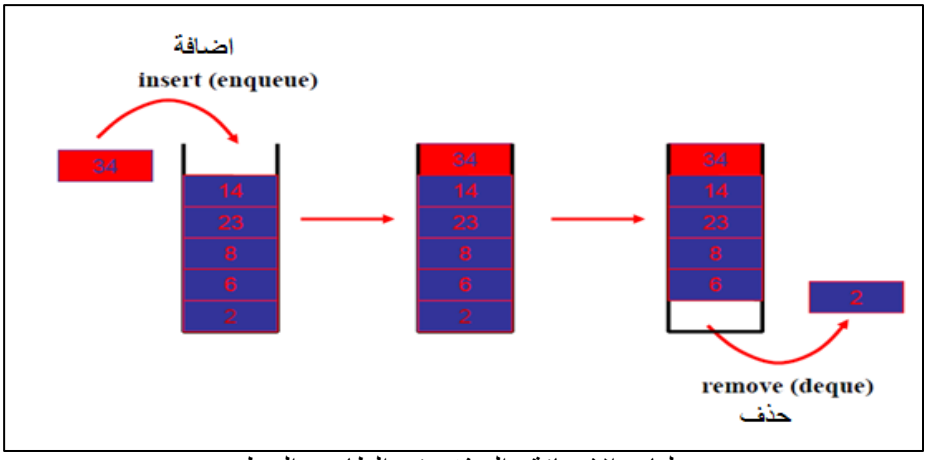
يمكن تمثيلها كمصفوفة وتدعى حينها هياكل البيانات الثابتة static data structure وعندها سترث جميع صفات المصفوفات، أو من الممكن ان يتم تمثيلها كقائمة موصولة وبهذا تدعى هياكل البيانات الديناميكية dynamic data structure وستمتلك جميع خواص وصفات القوائم الموصولة، وبغض النظر عن طريقة تمثيل الطابور فإن عمليات الاضافة تحدث في نهاية الطابور بينما يتم الحذف عند رأس الطابور. ويحتوي الطابور على مؤشرين أحدهما مؤشر البداية والآخر مؤشر النهاية.



الهيكل العام للطابور الخطي

يتميز تمثيل الطابور باستخدام المصفوفة بأنه سهل التنفيذ وكذلك معرفة حجم الطابور عند التعامل معه. أما **العيوب** فتتمثل في أنه قد لا يتم استخدام المساحة المحجوزة بشكل جيد مما يسبب في فقدان مواقع الذاكرة عند استخدام عدد قليل من العناصر مع الاعلان عن -حجز- مساحة كبيرة للطابور. ومن عيوب هذا النوع من التمثيل كذلك، أنه لا يمكن إضافة عناصر جديدة أكثر من عدد العناصر التي حدد بها حجم الطابور (حجم المصفوفة) مسبقاً.

وعند تمثيل طابور من النوع الساكن (محدود الحجم) فيتم استخدام متغير من النوع الصحيح ولنسمه max مثلاً ويرمز هذا المتغير إلى حجم أو سعة الطابور. وهناك ثلاث عمليات أساسية يمكن تطبيقها على الطابور وهي الإضافة إلى نهاية الطابور وعملية الحذف من بداية الطابور وكذلك عملية اختبار ما إذا كان الطابور فارغاً أم لا قبل كل عملية إضافة أو حذف.



عمليات الاضافة والحذف في الطابور الخطي

خوارزمية اضافة عنصر الى الطابور الساكن

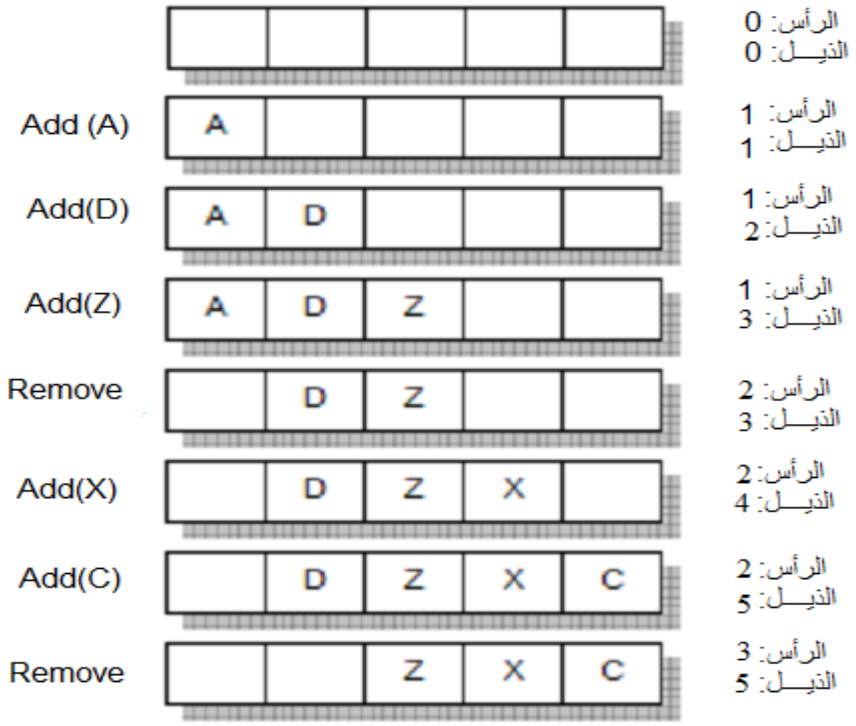
1. يتم اولا فحص مؤشر النهاية rear للطابور، إذا كان يوشر على الموقع الأخير للطابور (وفقاً لحجم الطابور المحدد مسبقاً)، فيتم إرسال رسالة بان الطابور ممتلئ ولا يوجد به فراغ.
2. اذا كان مؤشر النهاية يساوي 0 عندها يتم زيادة مؤشر النهاية ومؤشر البداية بمقدار واحد (لان ذلك يعني ان مؤشر البداية أيضاً يساوي 0، أي أن front=rear=0)، أي أن الطابور فارغ ولا يحتوي على أي عنصر.
3. إذا لم يكن مؤشر النهاية rear يؤشر على الموقع الأخير للطابور, عندها يتم ما يلي:
   a. تغيير موقع مؤشر النهاية بإضافة واحد له rear++ (أي ينتقل المؤشر الى الموقع الجديد).
   b. يضاف العنصر الجديد.

خوارزمية حذف عنصر من الطابور

1. يتم فحص مؤشر البداية للطابور, إذا كانت قيمته تساوي 0، يتم ارسال رسالة بأن الطابور فارغ ولا يمكن إجراء عملية الحذف.
2. إذا كان مؤشر البداية للطابور يؤشر على أحد العناصر (أي أن قيمته لا تساوي 0)، فيتم حذف العنصر الذي يوشر علية مؤشر البداية كما يلي:

a. يتم تغيير موقع المؤشر بزيادته بمقدار واحد (إن مجرد تغيير قيمة المؤشر بزيادته بواحد يعني ان العنصر تم حذفه).

b. إذا كان مؤشر البداية يساوي مؤشر النهاية ولايساوي صفر، فإن ذلك يدل على أن الطابور يحتوي على عنصر واحد فقط، عندها يتم تصفير المؤشران (تسند لهما القيمة 0) حيث أن الطابور سيصبح فارغاً بعد أن تم حذف آخر عنصر من عناصره.



عمليات الاضافة والحذف في طابور خطي

ويتبين مما سبق أن الطابور يمكن أن يكون في إحدى الحالات التالية:

ت	حالة الطابور	الوصف
1	front = rear	يعني ذلك أن الطابور يحتوي على عنصر واحد فقط بشرط ألا تكون قيمة أي من front أو rear تساوي صفر.
2	front = rear =0	في هذه الحالة يكون الطابور فارغاً ولا يمكن إجراء عملية الحذف منه، أي أن empty=true
3	rear < front أو front > rear	يتم التحقق من هذا الشرط عند حذف عنصر من الطابور، حيث يتم زيادة المؤشر front بمقدار 1، عندها يصبح rear أقل من front أو بمعنى آخر أن front أكبر من rear يدل ذلك على أن الطابور أصبح فارغاً. يتم بعدها اسناد القيمة 0 لكل من front و rear.
4	rear < max	إذا كانت ساعة الطابور محدودة فإنه يمكن الإضافة إلى الطابور فقط عنده تحقق هذه الحالة وهذا الشرط.
5	rear = max	يعني ذلك أن الطابور ممتلئ، ولا يمكن إضافة عناصر جديدة إليه. وتعرف عملية محاولة الإضافة إلى طابور ممتلئ بحالة Overflow
6	front < rear	يعني ذلك أن الطابور يحتوي على أكثر من عنصر ويمكن إجراء عملية الحذف منه، وأن empty=false

ولمعرفة عدد العناصر الموجودة في الطابور في أي وقت وعند أي حالة يتم استخدام العلاقة أو المعادلة التالية: items_number = (rear - front) +1

برنامج للتعامل مع الطابور الخطي وتنفيذ عمليتي الإضافة والحذف

```
#include <iostream>
#define max 10
using namespace std;

// تعريف المتغيرات العامة للطابور
int q[max];
int Front = 0;
int Rear = max - 1;
int counter = 0;

// دالة لإدخال عنصر إلى الطابور
void enqueue(int x) {
    Rear = (Rear + 1) mod max;
    q[Rear] = x;
    counter++;
}

// دالة لحذف عنصر من الطابور
int dequeue() {
    int x = q[Front];
    Front = (Front + 1) mod max;
    counter--;
    return x;
}

// دالة للحصول على حجم الطابور الحالي
int getSize() {
    return counter;
}

int main()
{
    enqueue(1);
    enqueue(2);
    enqueue(3);

    while (getSize())
     {
        cout << dequeue() << endl;
     }

    return 0;
}
```

يمكن كتابة البرنامج بصيغة أخرى (لاحظ المتغير front, rear)

```
#include <iostream>
#define max 10
using namespace std;

// تعريف المتغيرات العامة للطابور
int q[max];
int Front = 1;
int Rear = 0;
int counter = 0;

// دالة لإدخال عنصر إلى الطابور
void enqueue(int x) {
    Rear = (Rear + 1);
    q[Rear] = x;
    counter++; }

// دالة لحذف عنصر من الطابور
int dequeue() {
    int x = q[Front];
    Front = (Front + 1);
    counter--;
    return x; }

// دالة للحصول على حجم الطابور الحالي
int getSize() {
    return counter; }

int main()
{
    enqueue(1);
    enqueue(2);
    enqueue(3);

    while (getSize())
     {
        cout << dequeue() << endl; }
    return 0;
}
```

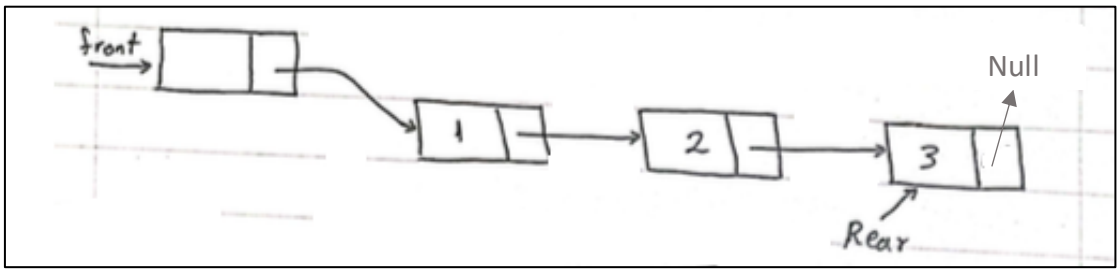
تمثيل الطابور بهياكل بيانات ديناميكية (طابور متغير الحجم)

يتم في هذه الحالة تمثيل الطابور باستخدام القوائم الموصولة. وكما هو الحال مع القوائم الموصولة، فان كل عقدة في الطابور تحتوي على حقل بيانات وحقل مؤشر يشير الى عنوان العقدة اللاحقة .

وعند تمثيل الطابور نحتاج الى مؤشرين أحدهما يوشر الى العقدة الأولى (بداية الطابور front) ومؤشر آخر يؤشر إلى العقد الأخيرة (نهاية الطابور rear). الشكل اللاحق يوضح طريقة الخزن لطابور إسمه queue كل عقدة تتكون من حقل بيانات DataItem ومؤشر NextNode يشير إلى العقدة اللاحقة. يمكن أن يكون مؤشر بداية الطابور عقدة فارغة لا تحتوي على بيانات وتكون هي العقدة الأولى في الطابور، ويتم استخدامها أحيانا في معالجة بعض الحالات الخاصة، فيمكن مثلا تخزين عدد العقد أو العناصر في

الطابور في حقل البيانات لهذه العقدة حيث يتم تحديثها عنده إجراء أي عملية إضافة أو حذف إلى الطابور الموصول.

تتميز عملية تنفيذ الطابور باستخدام القوائم الموصولة بأنه يتم تخصيص ذاكرة بشكل آلي لكل عنصر جديد. وتعتبر طريقة جيدة لترشيد استهلاك الذاكرة. أما **عيوب** هذا النوع من التمثيل فهي صعوبة التعامل مع الطابور ومتابعة مؤشرات بداية ونهاية الطابور.



تمثيل الطابور الخطي باستخدام القوائم الموصولة

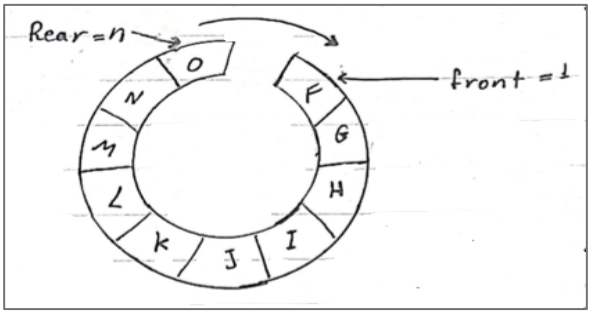
برنامج C++ لتمثيل الطابور على شكل قائمة موصولة

```
#include <iostream>
using namespace std;

struct node
  {
    int data;
    node* next;
  };
typedef node* node_ptr;

node_ptr front = Null;
node_ptr rear = Null;

// دالة إضافة عنصر إلى الطابور
void enqueue(int value)
   {
    node_ptr newNode = new node;
    newNode->data = value;
    newNode->next = Null;

    if (front == Null)
        front = rear = newNode; // أول عنصر
      else
       {
        rear->next = newNode;
        rear = newNode;
       }
    cout << "تمت إضافة العنصر: " << value << endl;
   }

// دالة حذف عنصر من الطابور
void dequeue()
   {
```

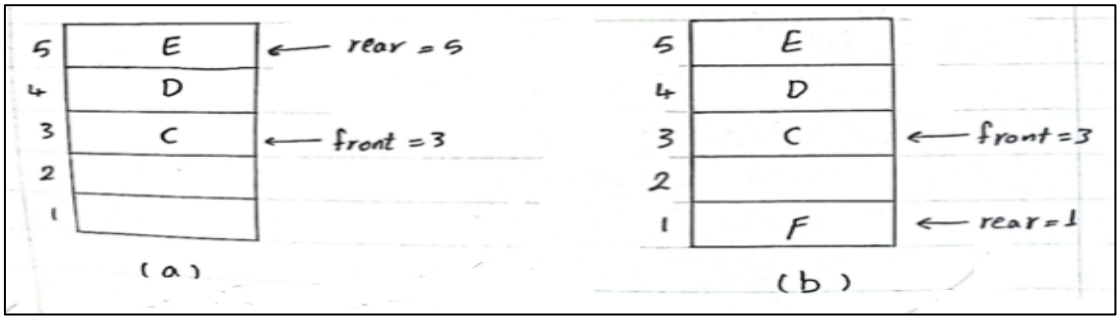
```
    if (front == Null)
      {
        cout << "الطابور فارغ! لا يمكن الحذف.\n";
        return;
      }

    node_ptr temp = front;
    front = front->next;

    if (front == Null)
        rear = Null;
     cout << "تم حذف العنصر: " << temp->data << endl;
     delete temp;
   }

// دالة لعرض عناصر الطابور
void display()
   {
    if (front == Null)
      {
        cout << "الطابور فارغ.\n";
        return;
      }
    cout << "عناصر الطابور: ";
    node_ptr temp = front;
    while (temp != Null)
      {
        cout << temp->data << " ";
        temp = temp->next;
      }
    cout << endl;
   }

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();

    dequeue();
    display();

    return 0;
}
```

الطابور الدائري

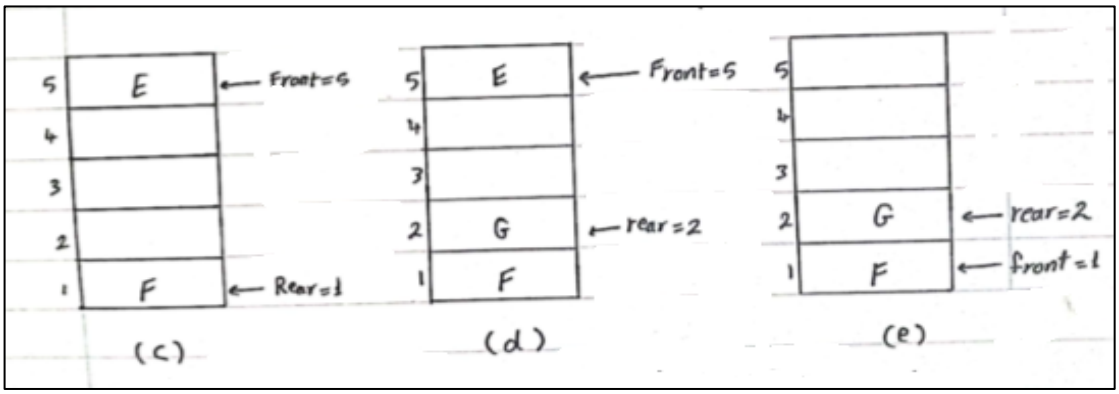
إن عملية تمثيل الطابور كمصفوفة (كما تم توضيحه سابقاً) قد يسبب ضياعاً في مواقع الذاكرة خصوصاً عندما يصل مؤشر الإضافة rear إلى نهاية الطابور مع وجود مواقع شاغرة في بداية الطابور بسبب عمليات الحذف التي تمت في مقدمته. ولتفادي هذه المشكلة، ومن أجل إعادة استغلال المواقع الشاغرة في مقدمة الطابور فإنه يتم استخدام الطابور الدائري حيث يأتي أول عنصر في الطابور مباشرة بعد آخر عنصر فيه ويمكن أن نتصور ذلك كما في الشكل التالي:



وذلك يعني أنه في حال أن الموقع الأخير من الطابور مشغول فإنه يمكن إضافة عنصر جديد بعد آخر عنصر - أي في بداية الطابور- طالما أن الموقع في بداية الطابور شاغراً، ولنأخذ المثال التالي لتوضيح ذلك. نفترض أن الطابور يحتوي على ثلاثة عناصر كما هو مبين في الشكل a:



على الرغم من أن آخر موقع في الطابور مشغول، إلا انه يمكن إضافة عناصر أخرى إلى الطابور طالما أنه توجد مواقع شاغرة في بدايته. فإذا أردنا الآن إضافة عنصر إلى الطابور، فإن ذلك يتم عن طريق إضافة هذا العنصر في الموقع رقم 1 من الطابور كما هو مبين في الشكل b. بذلك يكون أول عنصر في الطابور موجود في الموقع رقم 3 بينما يكون آخر عنصر موجود في الموقع رقم 1. وتبين الاشكال التالية e, d, c حالة الطابور عندما يتم حذف عنصرين منه وهما D, C ثم يضاف بعد ذلك العنصر G، وأخيرا يتم حذف العنصر E.



باستخدام هذا التمثيل يكون من الصعب تحديد متى يكون الطابور فارغاً، فالشرط front > rear والمستخدم سابقاً للتعبير عن حالة أن الطابور فارغاً لم يعد صالحاً للتطبيق هنا. فعلى الرغم من أن الاشكال أو الحالات b,c,d يكون الشرط السابق فيها صحيحاً، أي أن front > rear إلا أن الطابور ليس فارغاً في أي منها. لتفادي هذه المشكلة فإنه يتم اعتبار أن المؤشر **front سيشير إلى العنصر الذي يسبق أول عنصر في الطابور** بدلاً من أن يشير إلى العنصر الأول نفسه، ويستخدم الشرط front=rear للتعبير على أن الطابور فارغاً ويمكن الإعلان عن طابور دائري من الأعداد الصحيحة كما يلي:

```
#include <iostream>
using namespace std;
const int max = 10;
int items[max];

int front = max; // المؤشر الذي يسبق أول عنصر في الطابور
int rear = max; // يشير إلى آخر عنصر في الطابور
```

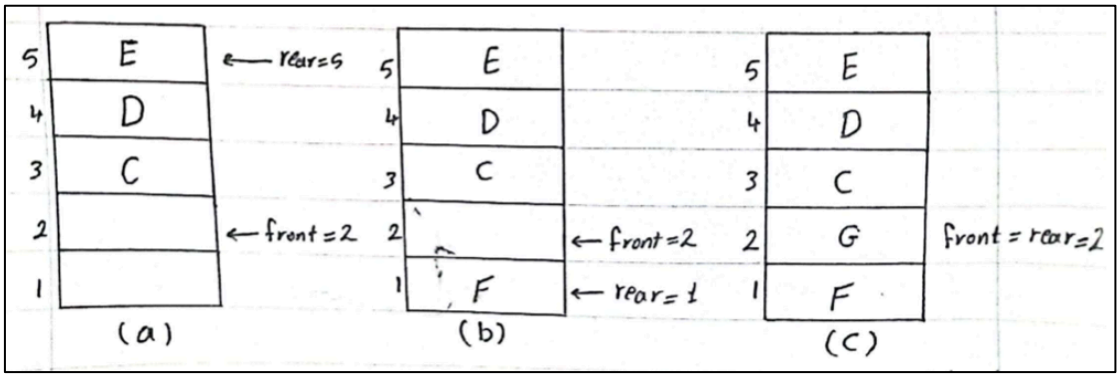
لاحظ أن القيمة الابتدائية للمؤشرين front و rear تساوي 10 وهي آخر موقع في الطابور وذلك لأن آخر موقع أو عنصر في الطابور يسبق مباشرة أول عنصر فيه. وبما أن front=rear فهذا يعني أن الحالة الابتدائية للطابور أنه فارغاً. ويمكن استخدام الدالة التالية لاختبار حالة ما إذا كان الطابور فارغاً أم لا.

```
bool isEmpty() {
    return front == rear;
}
```

وعند حذف عنصر من الطابور يتم تحديث قيمة المؤشر front كما يلي - front = (front + 1) mod max ويمكن كتابة دالة حذف عنصر من الطابور الدائري كما يلي:

```
void dequeue()
 {
   if (isEmpty())
      {
       cout << " الطابور فارغ" << endl;
       front = max;
       rear = max;
      }
   else
       {
        Cout << "تم حذف العنصر" <<items[front];
        front = (front + 1) mod max;
       }
}
```

وعند إضافة عنصر إلى الطابور يجب التحقق من أن الطابور غير ممتلئ قبل إجراء عملية الإضافة وذلك لتفادي حالة الـ Overflow. ولنأخذ المثال التالي لتوضيح عملية إضافة عنصر إلى الطابور الدائري، مع ملاحظة أن المؤشر front يشير إلى العنصر الذي يسبق أول عنصر في الطابور.



في الشكل a يحتوي الطابور على ثلاثة عناصر وهي E, D, C في المواقع 5,4,3. لاحظ أن rear=5 وأن front=2 ومن خلال الشكلين c و b نلاحظ أنه تمت إضافة العنصريين G وF إلى الطابور مما أدى إلى تغيير قيمة المؤشر rear, وعند هذه النقطة يكون الطابور ممتلئ وأن محاولة الإضافة ستؤدي إلى حالة Overflow .

ونلاحظ هنا أيضا بأن حالة امتلاء الطابور تمت الإشارة إليها من خلال الحالة التي تتساوي فيها قيمة المؤشرين front وrear أي أن (front=rear) ويجب ألا ننسى أن هذه الحالة قد استخدمت سابقاً للتعبير عن حالة الطابور الفارغ، ويبدوا من خلال ذلك أنه لا توجد حالة نستطيع فيها أن نميز أو نفرق بين حالة الطابور الممتلئ والطابور الفارغ.

يمكن معالجة هذه الإشكالية من خلال التنازل عن موقع واحد ضمن هيكل الطابور وعدم استخدامه لتخزين أي عنصر. ووفقًا لهذا الأسلوب، فإن عدد العناصر التي يمكن تخزينها فعلياً في الطابور يساوي سعة الطابور -1. فمثلا إذا كان حجم الطابور= 10، فإن الحد الأقصى لعدد العناصر المسموح بتخزينها = 9 عناصر فقط.

```
void enqueue(int value)
  {
    if (isFull())
      {
        cout << "الطابور ممتلئ!" << endl;
        return;
      }

    if (isEmpty())
      {
        // أول عملية إدخال
        front = max;
        rear = 1;
      }
    else
        rear = (rear + 1) mod max;

    items[rear] = value;
  }
```

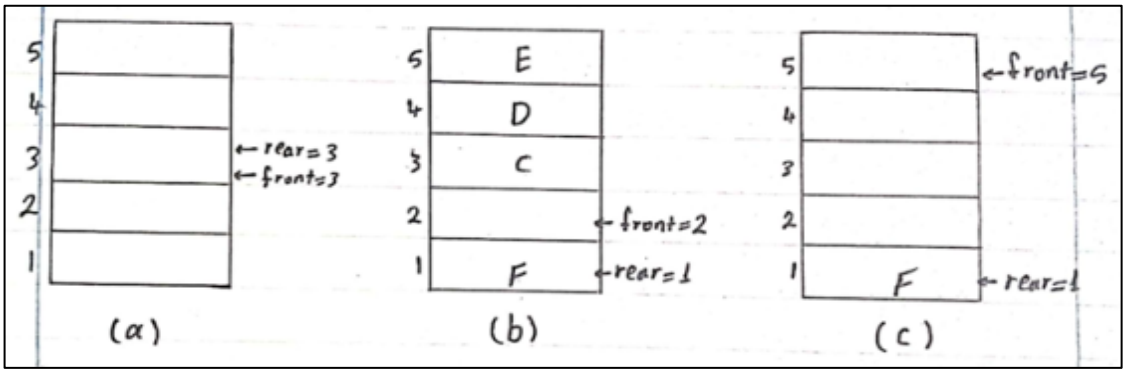
وتكون دالة التحقق من كون أن الطابور ممتلئ أم لا كما يلي:

```
bool isFull() {
    return (rear mod max) + 1 == front;
}
```

ويمكن تلخيص أهم الشروط التي يمكن تطبيقها على الطابور الدائري كما يلي:

1	front=rear	طابور فارغ
2	(rear mod max) +1 = front	طابور ممتلئ
3	(front mod max) +1 = rear	طابور يحتوي على عنصر واحد فقط

ولنأخذ المثال التالي لاختبار هذه الشروط.



- من الشكل a نلاحظ ان front=rear وهذا يدل على أن الطابور فارغ وهو متوافق مع الشرطة رقم 1 أعلاه.
- ومن الشكل b نلاحظ أن الطابور ممتلئ حسب الشرط السابق مع التنازل عن خانة واحدة من مساحة الطابور حسب الشرط رقم 2.

(rear mod max) +1 = front
(1 mod 5) +1 = 2
1 + 1 = 2
2 = 2

أما في الشكل c نلاحظ أن الطابور يحتوي على عنصر واحد فقط ويمكن التأكد من ذلك عن طريق تطبيق الشرط رقم 3 في الجدول أعلاه.

(front mod max) +1 = rear
(5 mod 5) +1 = 1
0 + 1 = 1
1 = 1